

Secure Password Manager (CLI-Based)

Professor Sami Saydjari
COSC 55 — Security and Privacy
Dartmouth College

Teaching Assistant: Mohamed Moustafa Dawoud

March 31, 2025

Abstract

This project proposes the development of a secure, command-line-based password manager as a course project for COSC 55 — Security and Privacy. The application will enable users to store and manage credentials locally using strong encryption, with access protected by a master password. The project will help demonstrate foundational principles of secure authentication, encryption, password storage, and usability within a security-first design.

Project Title

Secure Password Manager (CLI-based)

Overview

This project aims to build a command-line password manager that enables users to securely store and retrieve credentials for different services. The application will use symmetric encryption for data confidentiality and will require a master password to access the vault. It will help students understand core cybersecurity concepts such as encryption, secure storage, password hashing, and user authentication.

Objectives

- Implement secure storage of credentials using encryption.
- Allow users to create, retrieve, update, and delete password entries.
- Enforce authentication via a master password protected using password hashing.
- Build a user-friendly command-line interface for interacting with the password vault.

Learning Outcomes

By completing this project, students will:

- Gain hands-on experience with cryptography using Python.
- Learn how to handle sensitive data securely.
- Understand the differences between hashing and encryption.
- Design and implement basic authentication systems.

Technologies Used

- **Python 3.x**
- **cryptography** library (for symmetric encryption using Fernet)
- **hashlib** and **os.urandom** (for password hashing with salt)
- **json** or **sqlite3** (for data storage)

Proposed Features

- Master password registration and login (hashed + salted).
- Add new credentials (service, username, password).
- Encrypt and store credentials securely.
- Retrieve and decrypt credentials on demand.
- Search for credentials by service name.
- Edit and delete stored entries.
- Auto-lock after inactivity (optional).
- Random password generator (optional).

Security Design

- Passwords are encrypted using symmetric encryption (Fernet).
- Master password is hashed with salt using PBKDF2HMAC.
- No passwords are stored in plaintext.
- Credentials are only decrypted in memory during use.

Project Structure

```
password_manager/  
main.py           # CLI interface  
auth.py           # Master password setup and login  
crypto_utils.py   # Encryption and decryption logic  
storage.json      # Encrypted data store (or use SQLite)  
requirements.txt
```

Evaluation Criteria

- Secure implementation of password encryption and hashing.
- Functional CLI for interacting with password entries.
- Clean and modular code structure.
- Bonus points for optional features (e.g., auto-lock, password generator).